A Tutorial on using *Code::Blocks* with Catalina

BASIC CONCEPTS....2

PREREQUISITES....2

INSTALLING AND STARTING CODE::BLOCKS....3

Windows Installation....3
Linux Installation....4

USING CODE::BLOCKS....5

Changing your Catalina Project Options....16
Manually Specifying Catalina Options....19
Building a more Complex Catalina Project....22
Building The Catalina and Catalyst Demo Programs....26

NOTES ABOUT THE CATALINA TOOLS....27

General Note....27
If the Tools Menu is empty....27
If a program compiles and loads but won't run on the Propeller 1....27
The Build XMM Utilities Tool....27

ENVIRONMENT VARIABLES AND CODE::BLOCKS....28

CODE::BLOCKS LIMITATIONS....28

WHAT'S NEXT WITH CODE::BLOCKS?....28

Copyright 2013 Ross Higson

Basic Concepts

Like nearly all compilers, Catalina is essentially a command-line tool.

Fortunately, for Catalina users who don't like using command-line tools, or would just prefer to use an Integrated Development Environment (IDE), there are two IDEs available - one based on Geany - a lightweight yet fully functional graphical user interface, and one based on Code::Blocks - a professional graphical user interface that can be used as a front end for almost any C or C++ compiler.

The Catalina Geany IDE is now the recommended IDE for those who simply prefer to use an IDE over a command-line, but who are not familiar with Code::Blocks - it is simpler to learn and use. Code::Blocks is recommended only for those who are already familiar with this IDE, or who find that Geany is not sophisticated enough to manage the complexity of their C programs.

Code::Blocks provides a complete environment for developing C programs. It provides support for complex project structures, language sensitive editing, auto-completion, symbol management, intelligent search functions, automatic build capabilities and much more. Code::Blocks includes support for many different C compilers. This support is implemented via *plugins*.

Catalina provides an enhanced Code::Blocks Compiler plugin which adds support for the Catalina compiler. A project wizard for creating Catalina projects is also provided, as are a set of 'tool' wrappers that allow the use of various Catalina utilities (such as **payload** and **blackbox**) from within Code::Blocks.

This document provides details on installing and using Code::Blocks with Catalina. It is not a tutorial on either **C**, **Catalina** or **Code::Blocks**. It just provides enough information to get Code::Blocks up and running, and use it to compile Catalina programs.

Prerequisites

This document assumes you are installing Code::Blocks for use with Catalina release **4.0** (or later), and that you already have this version of Catalina installed in its default location. If you have not already installed Catalina, you should do that first - see the Catalina documentation for details.

If you have Catalina installed in a non-default location, the installation will still work correctly provided you set the environment variable **LCCDIR** to point to the main Catalina directory before starting the installation of Code::Blocks (otherwise you will have to manually configure Code::Blocks to tell it where Catalina is installed).

The current version of Catalina supports only Code::Blocks version **17.12**. Previous versions of Catalina supported Code::Blocks version 8.02 or 10.05 or 13.12, but if you have that version installed you will need to either upgrade it to version 17.12, or use the Catalina Windows installer and install another version of Code::Blocks (specifically for use with Catalina).

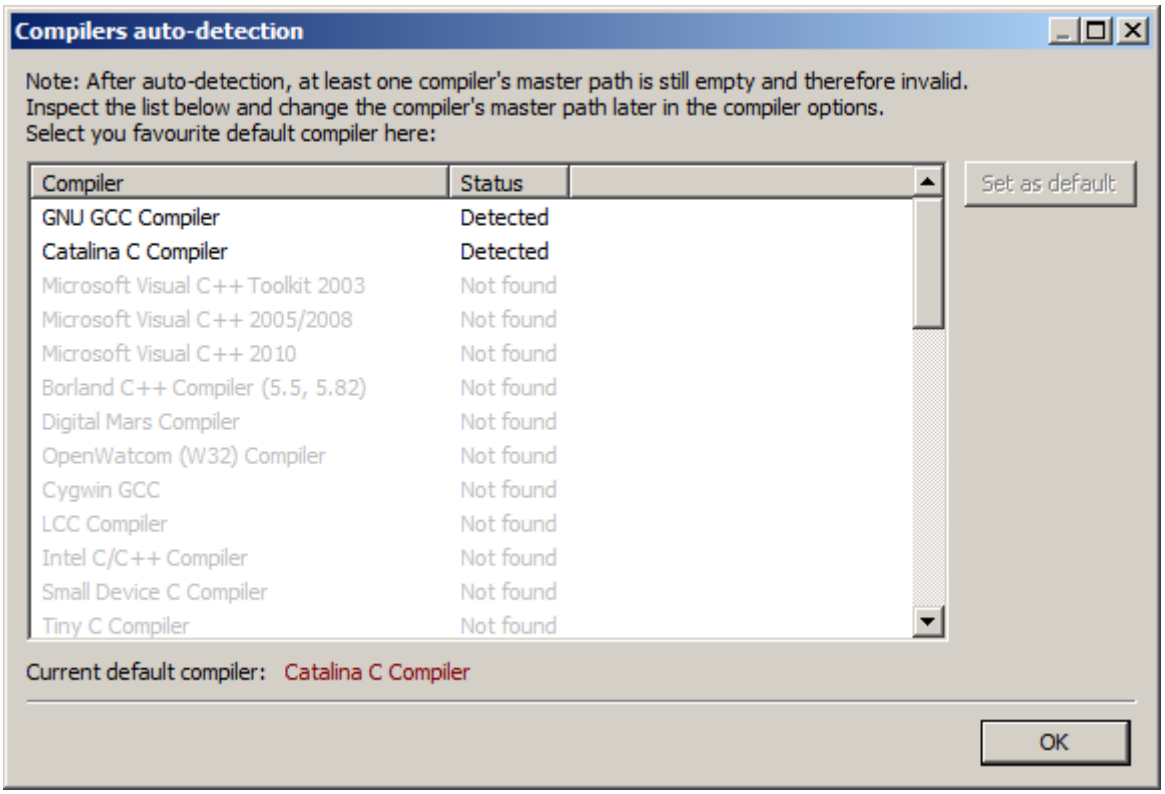
Like Catalina itself, Code::Blocks can be installed and used under either Linux or Windows. This document applies to both platforms, with any differences (mainly during the installation process) noted.

Installing and Starting Code::Blocks

Windows Installation

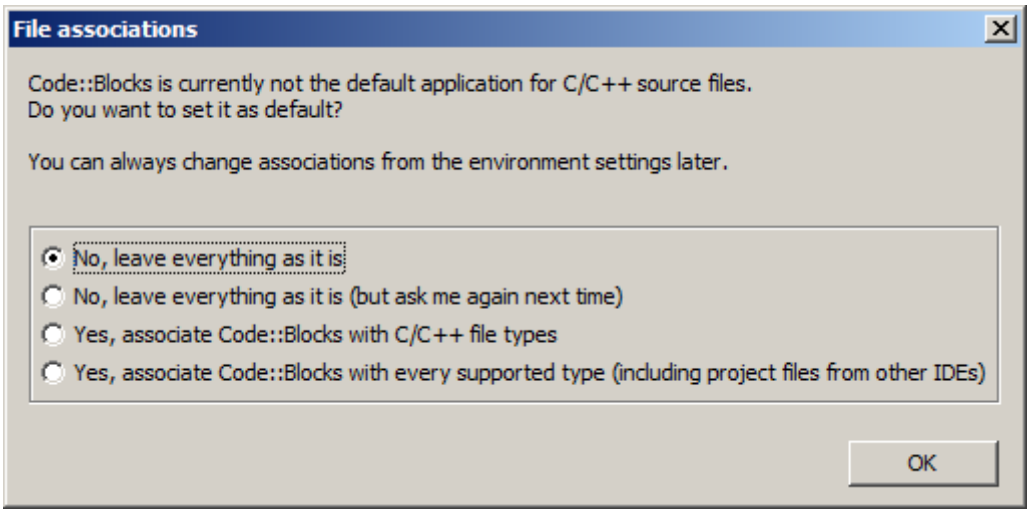
If you are using Windows, you must install Code::Blocks using the Windows “one touch” installer for Catalina, which includes a version of Code::Blocks built specifically for use with the Catalina compiler. To start Code::Blocks, use the **Catalina Codeblocks** start menu entry.

Note that when you start Code::Blocks for the first time, you may see a dialog box like this:



You should select the **Catalina C Compiler** from the list of compilers and press **OK**. To avoid having to do this every time you start Code::Blocks, you can press **Set as default** before pressing **OK**. You can also do this from within Code::Blocks later.

You may also see a dialog box asking if you would like to associate Code::Blocks with Code::Blocks workspace and project files:



You should select **Yes, associate Code::Blocks with every supported type**, otherwise you will not be able to open Code::Blocks Workspaces by selecting the Catalina start menu entries. Again, you can do this later from within Code::Blocks.

Linux Installation

A pre-compiled version of Code::Blocks for Linux is included in the distribution, but it must be manually unzipped and installed. It was built for Ubuntu 16.04 64-bit Linux, but may work on other versions and distributions.

The binaries are contained in the file *codeblocks_17.12_Linux.zip* in the Catalina *codeblocks* directory, and should be unzipped and installed to an appropriate location, such as */opt/codeblocks-17.12*

Then Code::Blocks can be started by executing:

```
/opt/codeblocks-17.12/bin/codeblocks
```

Note that you may need to set execute permissions on the Code::block binaries.

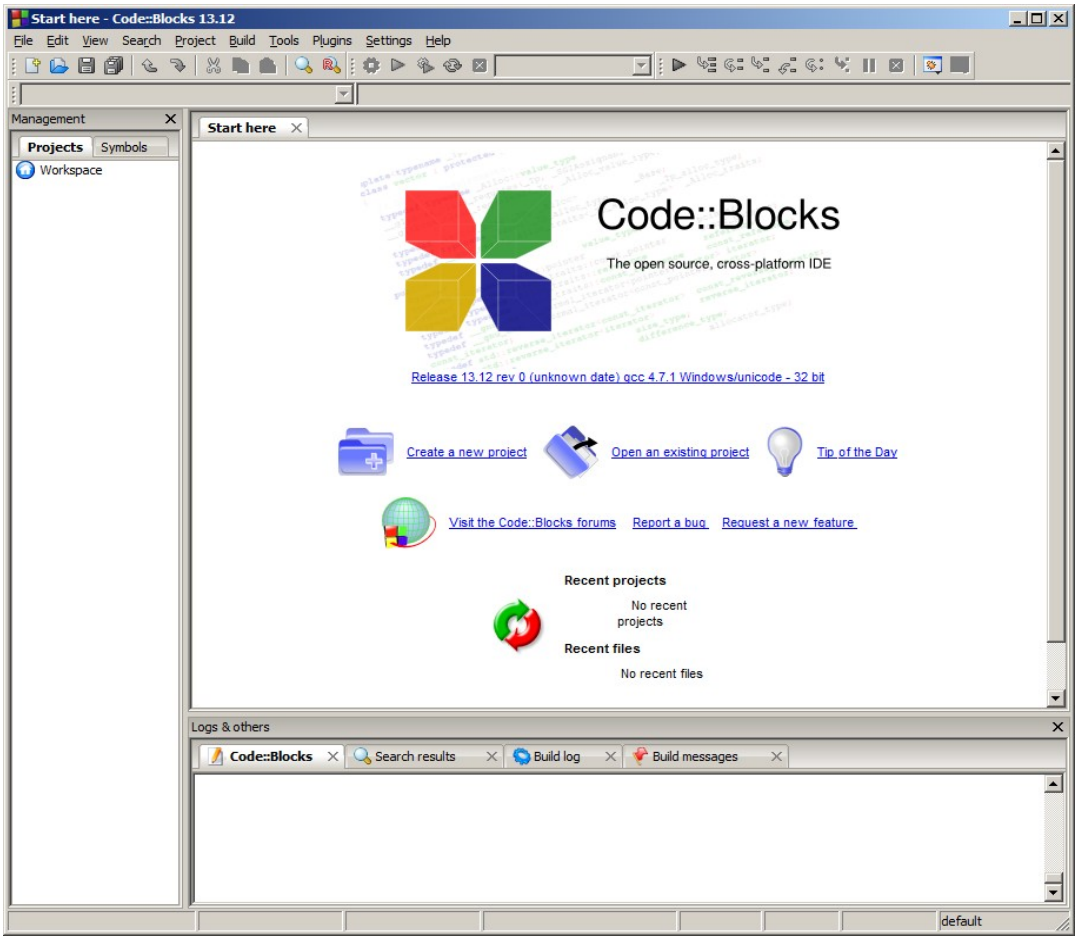
If the binaries provided do not work on your version of Linux, then you must build Code::Blocks from source. The source changes required for Catalina are included in the file *codeblocks_17.12_Src.zip*. Note that this file contains only the changes to the Code::Blocks source code – you must first download the source code for 17.12.

More details on building Code::Blocks from source are given in the file *README.Linux* in the main Catalina directory.

Note that on Linux, Code::Blocks assumes Catalina is installed in the default location of */opt/catalina* and if this is not the case then the auto-detection of the Catalina compiler may not work in Code::Blocks.

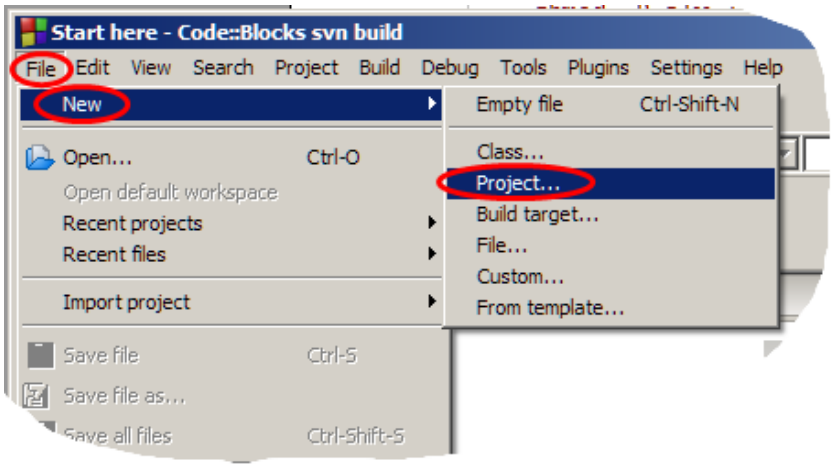
Using Code::Blocks

When you start Code::Blocks, the main window will look something like the following:

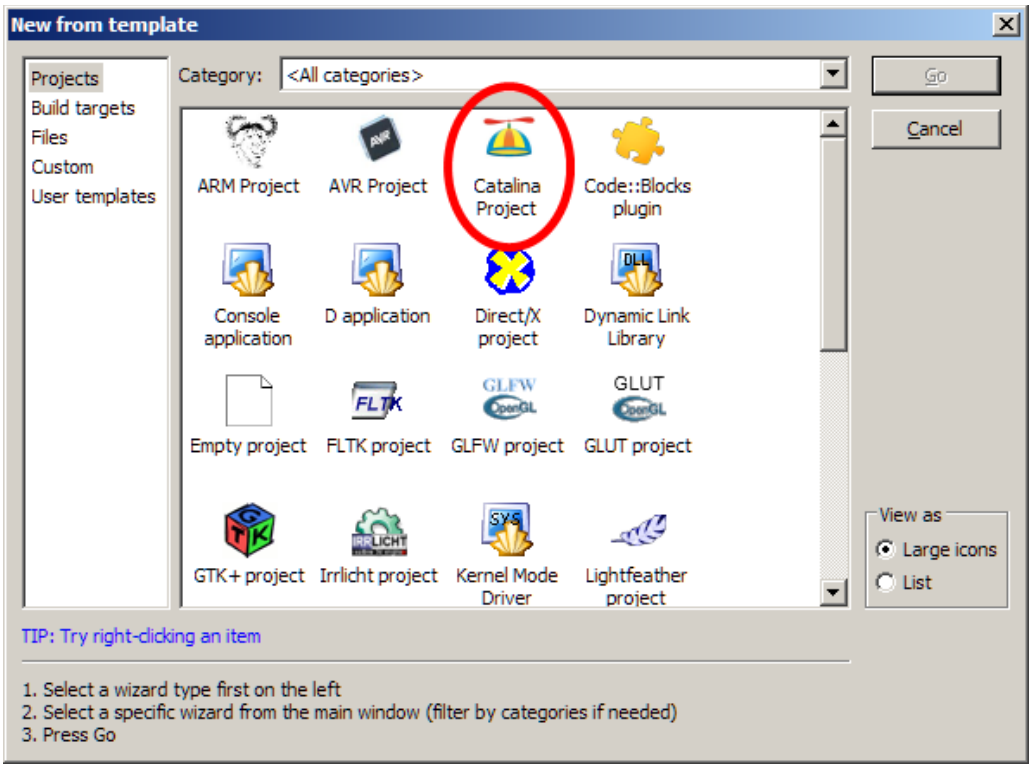


Creating your first Catalina Project

There are various ways to create a new project from within Code::Blocks, but the simplest way to get started is use the **Catalina Project Wizard**. To do this, select the entry **File->New->Project...** from the main menu:

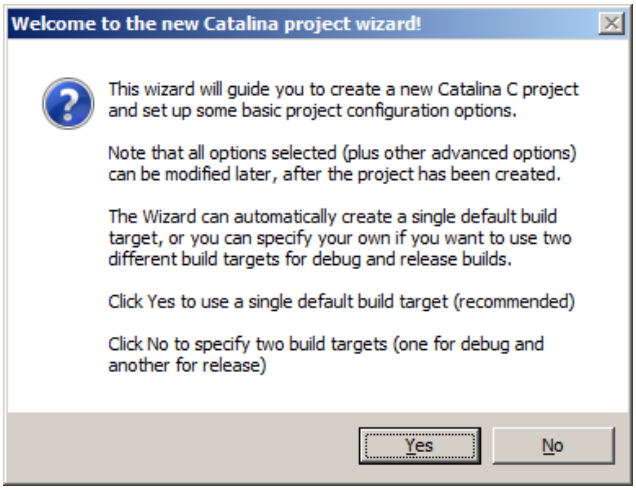


A dialog box similar to the one shown below will appear:

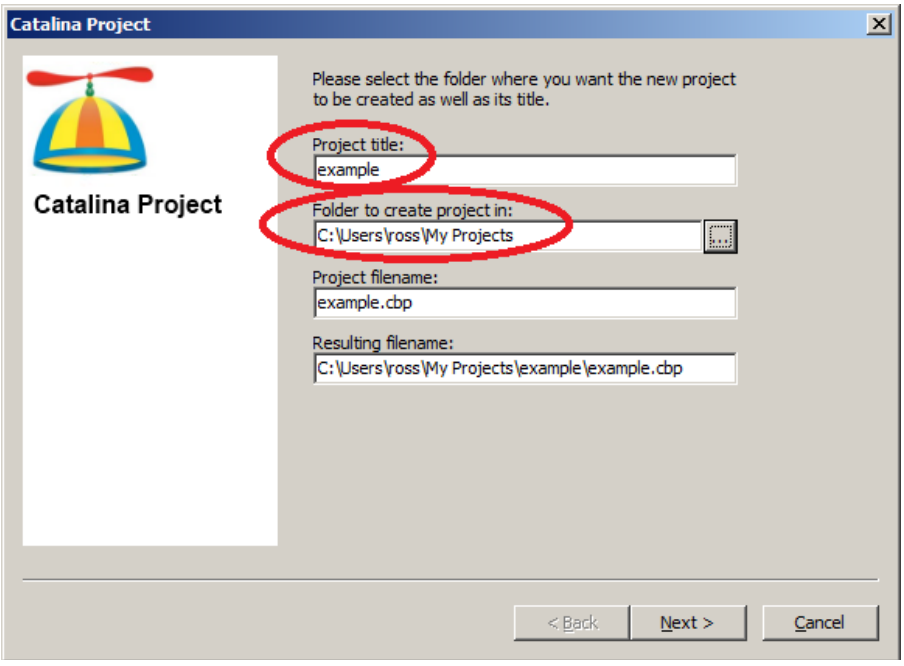


Select the **Catalina Project** Icon and press **Go**.

The Catalina Project Wizard will start, and display a welcome message. It will ask you to select your first option. This is whether you want a single default build target, or two build targets (one for debugging and one optimized for release):



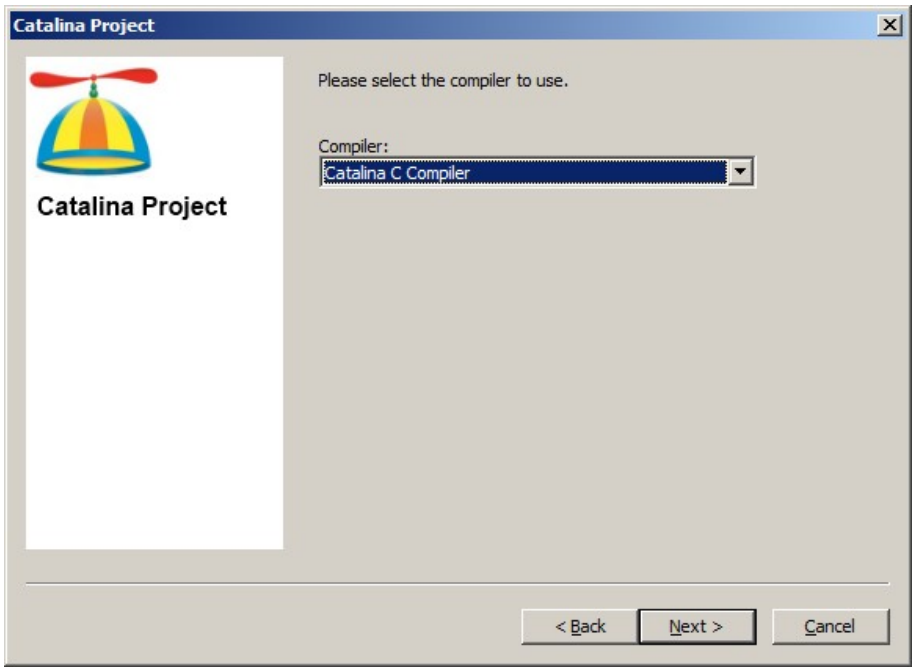
Let's not worry too much about what build targets are (you can read the Code::Blocks documentation later). For most purposes, we only need a single build target, so for now just press **Yes**



The Wizard will then display a page asking you for a title (name) for your project, and a directory for your project. Choose a suitable name for your project (don't use any embedded spaces, since this name will be used for the final binary or eeprom files) and where you want the directory for this project to be created (this can be anywhere, but you may choose to use the same directory for all your Catalina projects):

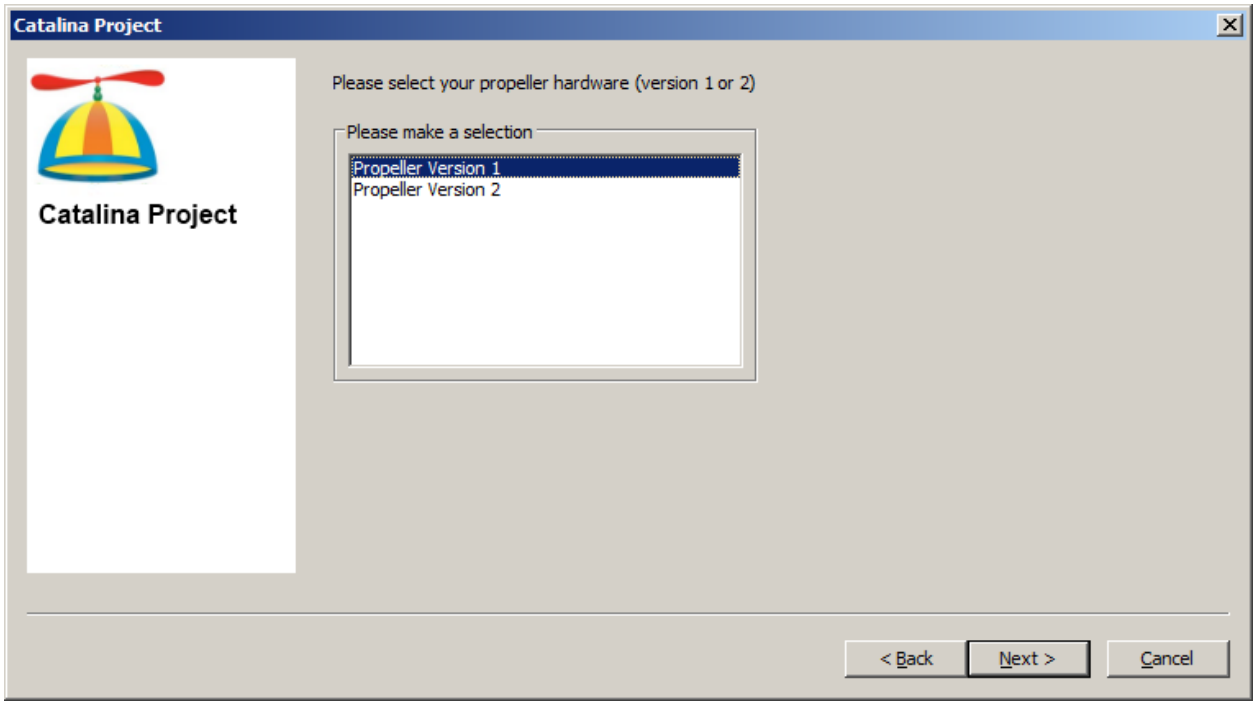
Then press **Next >**

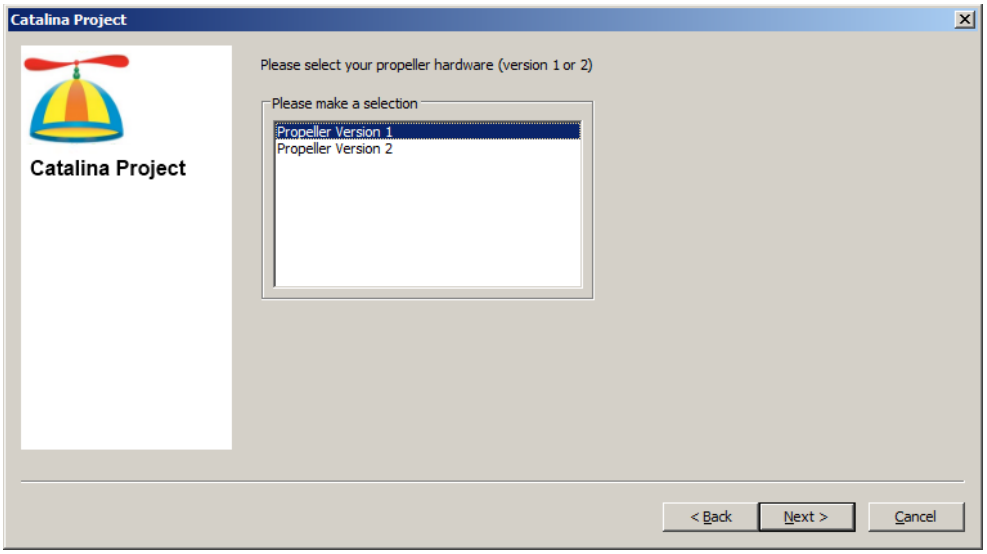
On the next page, Code::Blocks will give you the option to choose which Compiler to use:



The **Catalina C Compiler** should already be selected, and there is no reason to change anything here, so just press **Next >** again.

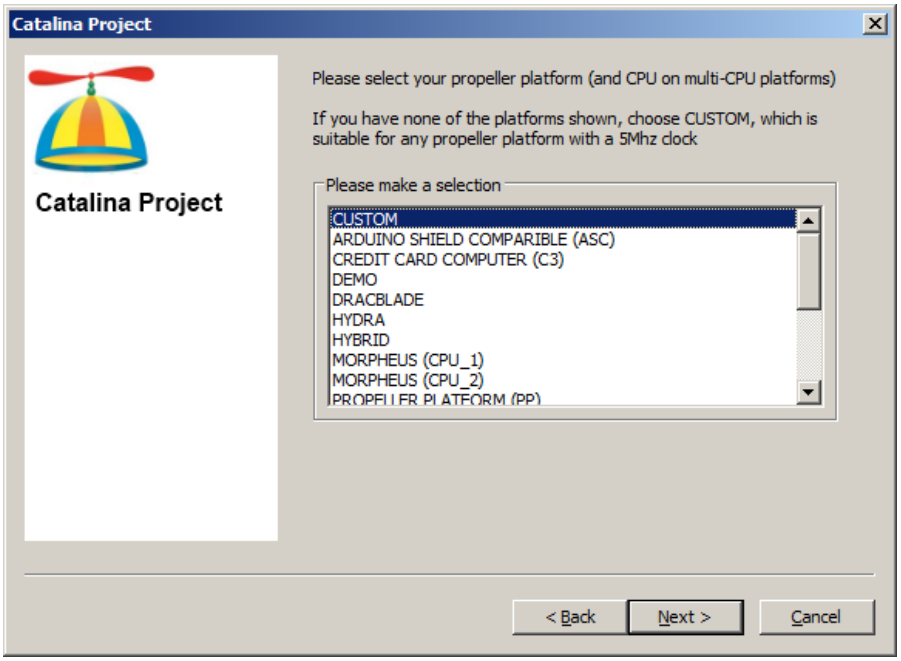
On the next page, things start to get interesting. Here you can choose the version of Propeller hardware you have. Currently supported are the Propeller 1 and the Propeller 2.





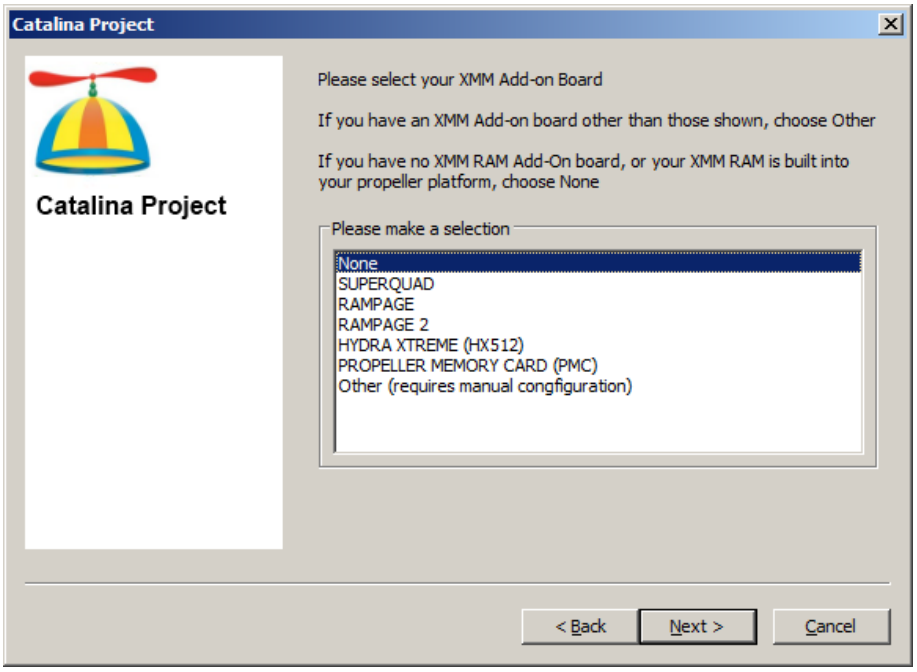
Select the appropriate hardware version, then press **Next >**

On the next page, you can choose the Propeller platform you have. Select the most appropriate entry from the list. If you are do not have any of the other platforms listed (note that some platforms have multiple entries, if they can be configured in various ways) then just leave **CUSTOM** selected. The **CUSTOM** platform is suitable for any Propeller that uses a 5Mhz clock (such as the Parallax QuickStart board and various Gadget Gangster boards).



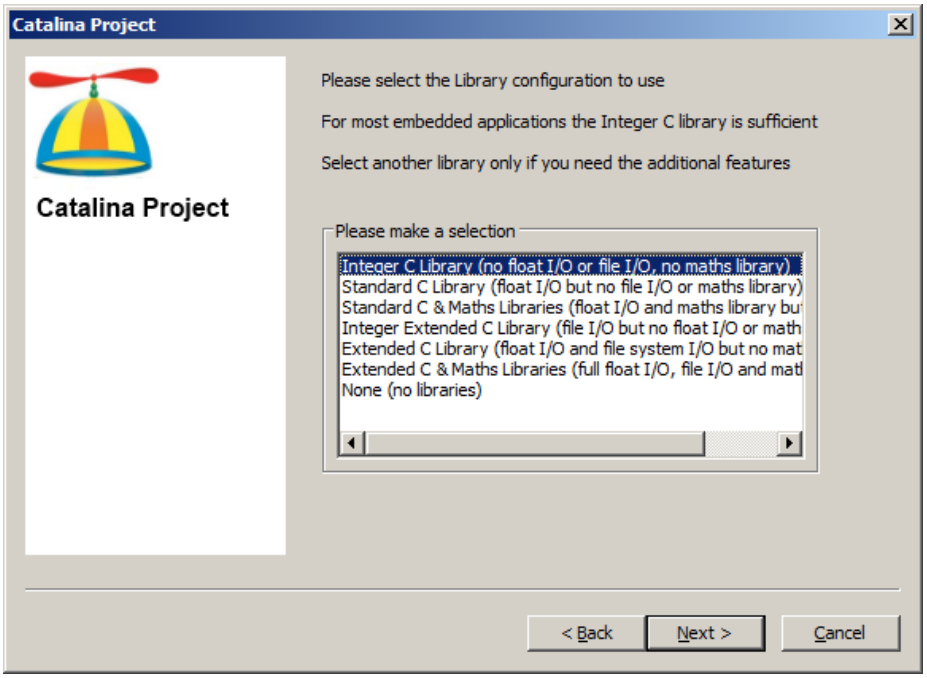
Then press **Next >**

On the next page, you can specify if you have an XMM Add-on board. If you have none, or your XMM is built-into the platform you chose in the previous step, then just select None.



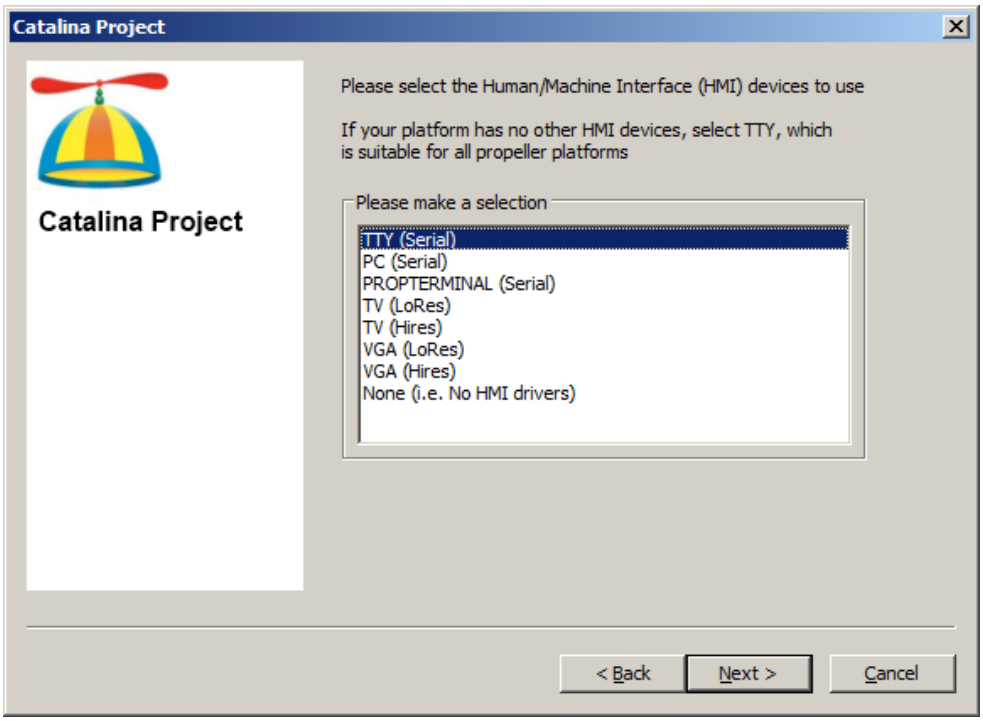
Then press **Next >**

On the next page, you can select what library configuration you want to use:



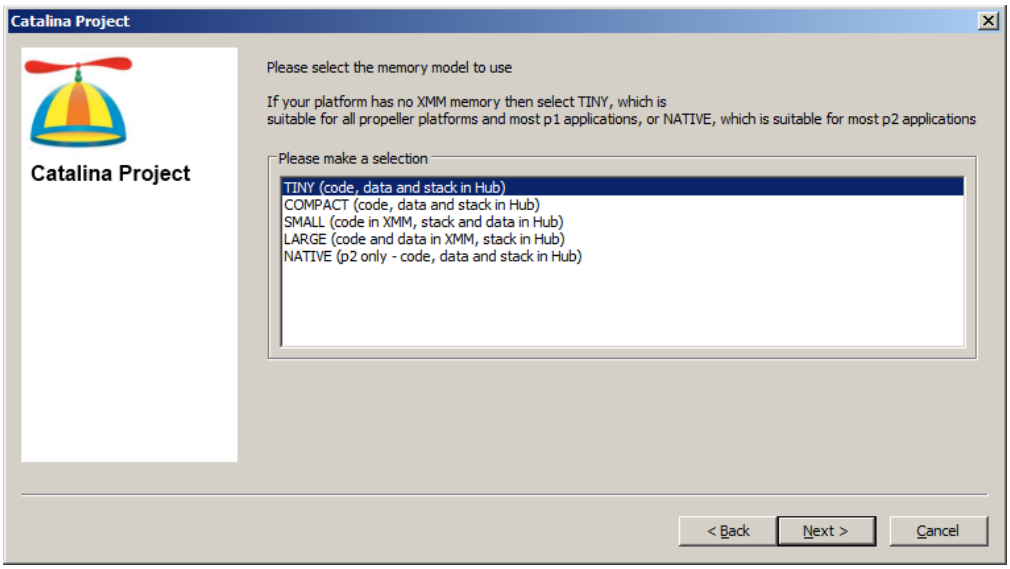
For the purposes of this example (and for most embedded applications), select the **Integer C Library** and then press **Next >**

On the next page, you will be able to select what Human/Machine Interface devices (i.e. what type of input and output devices) you want to use for this project:



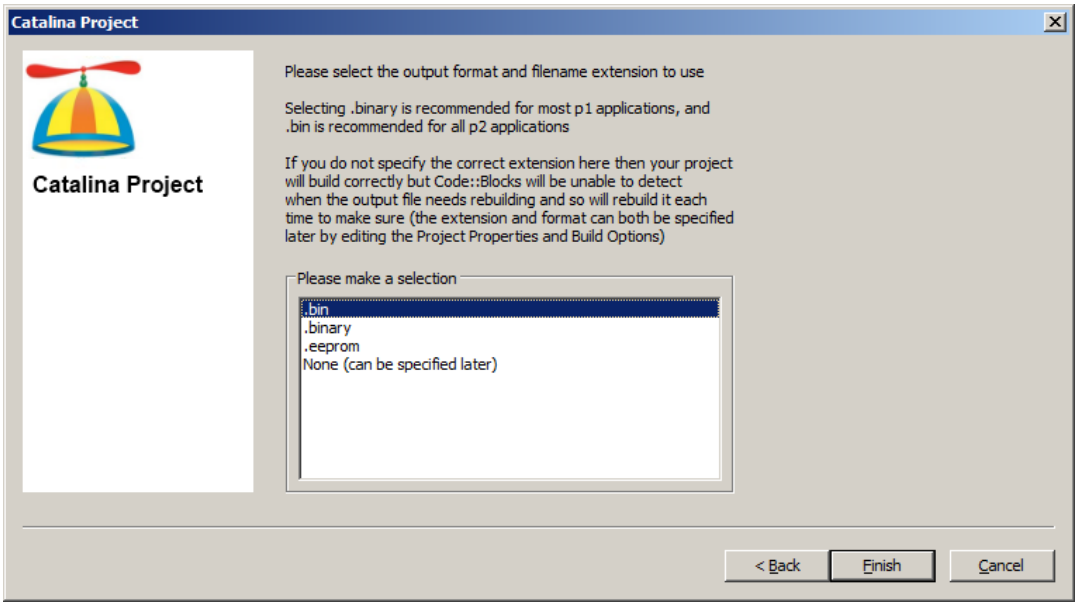
For the purposes of this example, select the **TTY (Serial)** option - this will use the Propellers default serial input and output capabilities, suitable for use with a terminal emulator program. Then press **Next >**

On the next page, you will be able to select what memory model you want to use for this project:



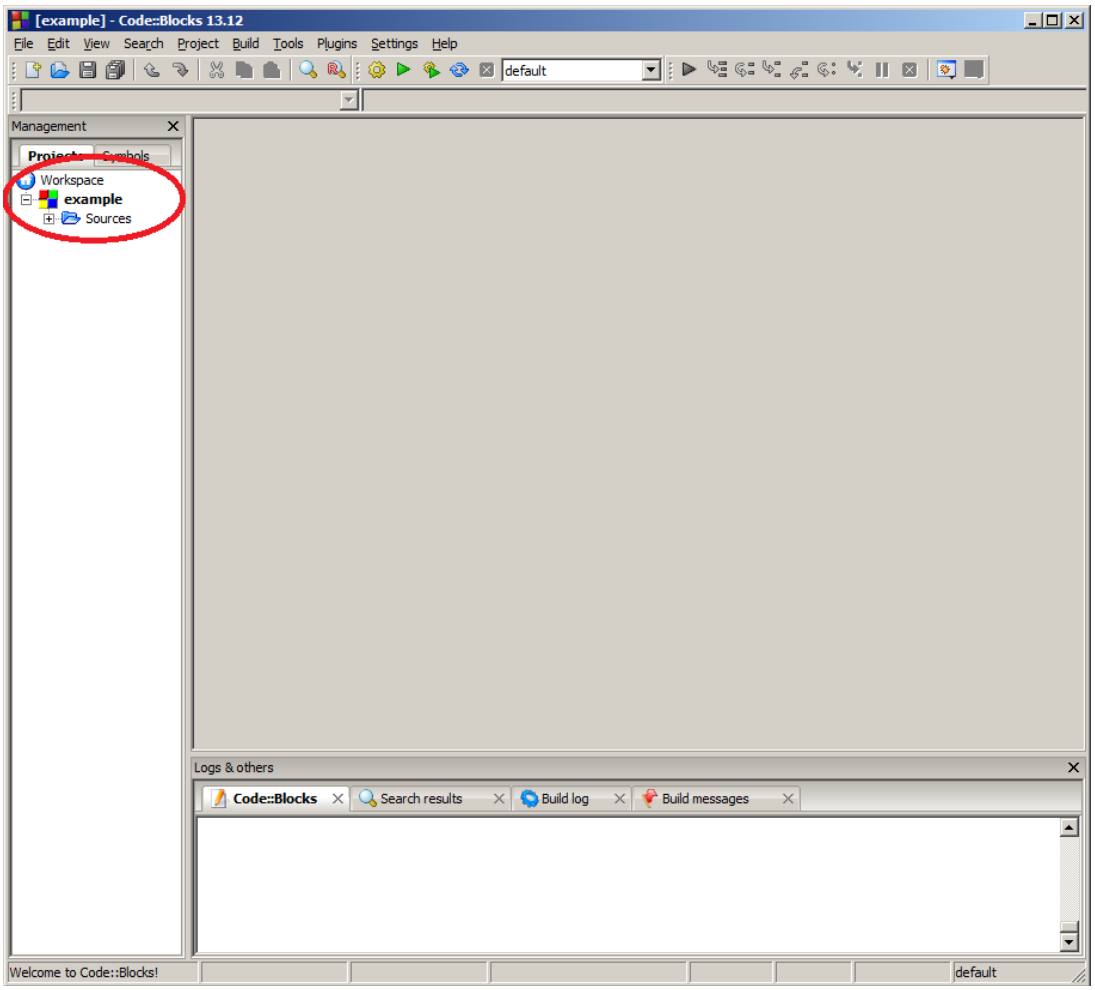
If you have a platform with XMM RAM, you can select any of the Catalina memory models here – but for the purposes of this example, just select the **TINY** or **COMPACT** memory model (which apply to all Propeller platforms) then press **Next >**

On the next page, you will be able to select what output format you want to use for this project:

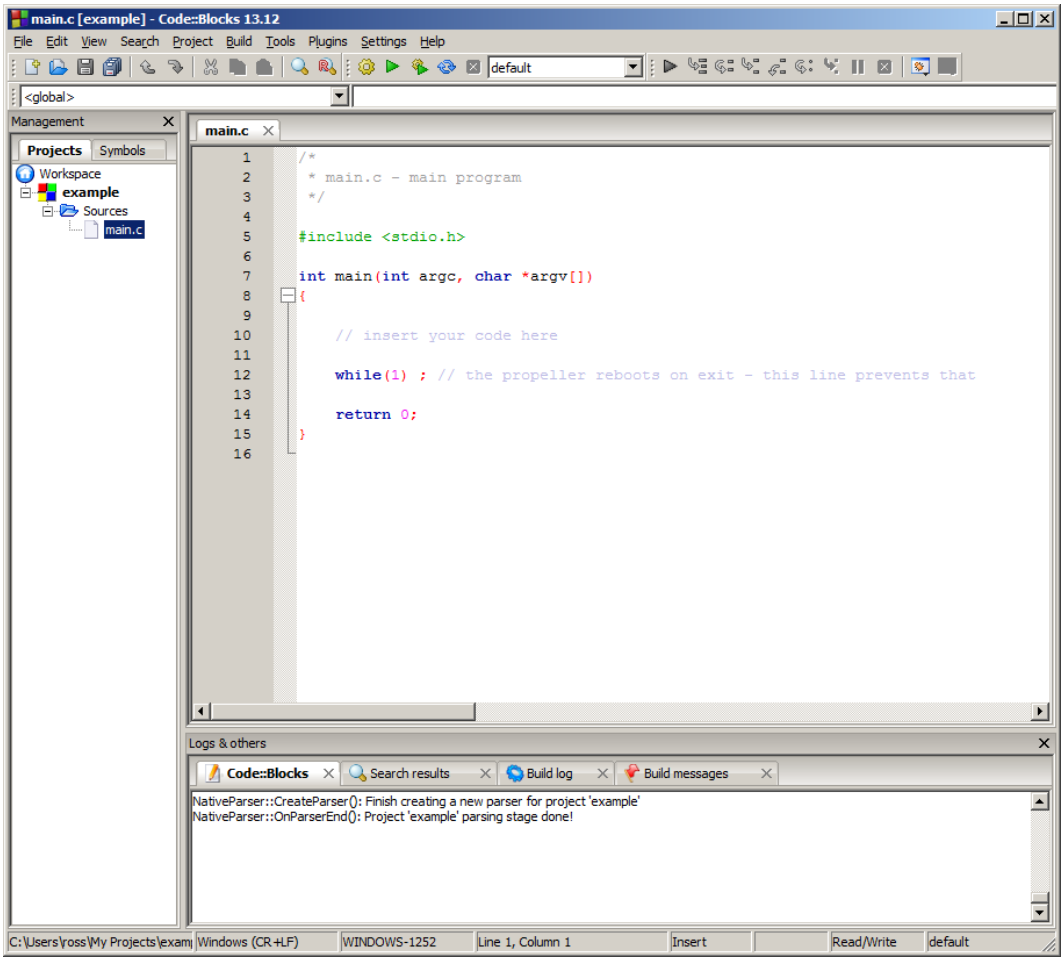


If you have a Propeller 2, you must select **.bin**. If you have a Propeller 1, then in most cases you will want to use a **.binary** format. Then press **Finish**.

Now Code::Blocks will create a project for you with the selected options. In the **Projects** tab of the Management pane, you will see your project:



In the project node you will see a node called **Sources**. Open this node and you will see that a file has been automatically created for your project called **main.c** – double click on this file to display the file in the source code editor window:



For the purposes of this example, lets simply modify a single line to this file in the text editor. Where the file contains the line:

```
// insert your code here
```

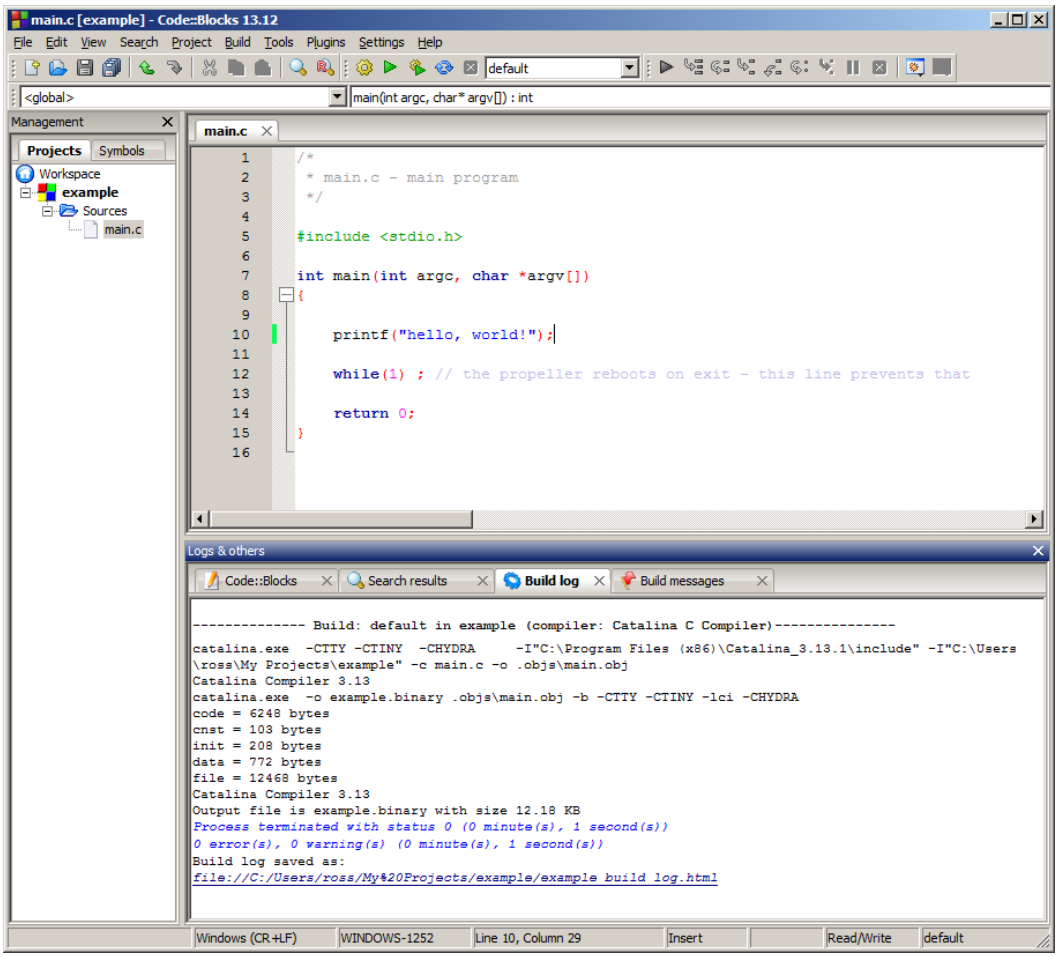
we will replace that with the line:

```
printf("hello, world!");
```

As you type this line, you will probably notice that Code::Blocks is anticipating what you intended - for instance, it enters a close bracket when you type an open bracket, and a second quote when typ type the first one - this can be confusing at first (and it can be disabled) - but it can also be very handy. Just make sure to keep typing until you have typed the above line - including the terminating "**;**".

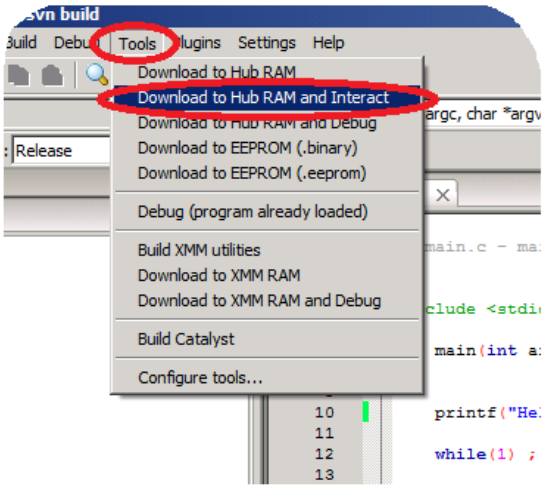
Now we can build this project. We can do this by pressing **Ctrl-F9**, or by selecting **Build->Build** from the menus, or by right clicking on the project in the navigation pane and selecting **Build**.

The output of the build will be shown in the build log pane (you can expand the pane by dragging the top of the pane upwards):

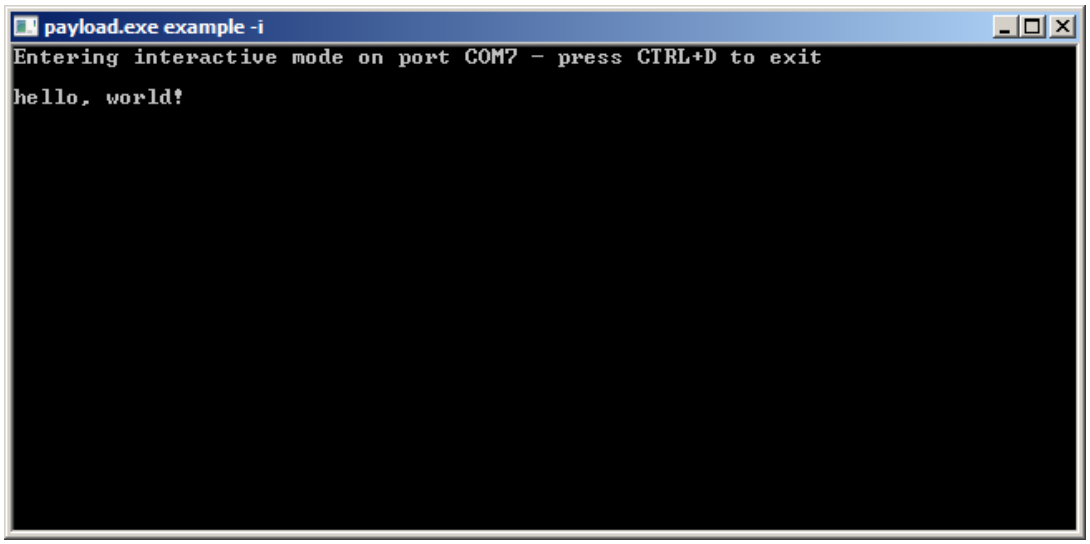


Assuming the project built correctly (which it should do if you typed the above line in correctly - otherwise correct any errors before continuing) then you can now do things with the project executable that has been generated by the build. You can do this by selecting various Catalina utilities from the **Tools** menu. Note that if the **Tools** menu is empty (except for a **Configure Tools …** entry) see the section later in this document titled **If the Tools Menu is empty**.

For the purposes of this example, if you have a Propeller 1 connected to your computer and it is powered on, select **Tools->Download to Hub RAM and Interact**, as shown below (or **Tools->Download to Hub RAM and Interact at 230400 baud** for the Propeller 2):



If everything works correctly, after a few seconds you should see your Propeller displaying a friendly greeting in a terminal emulator window:

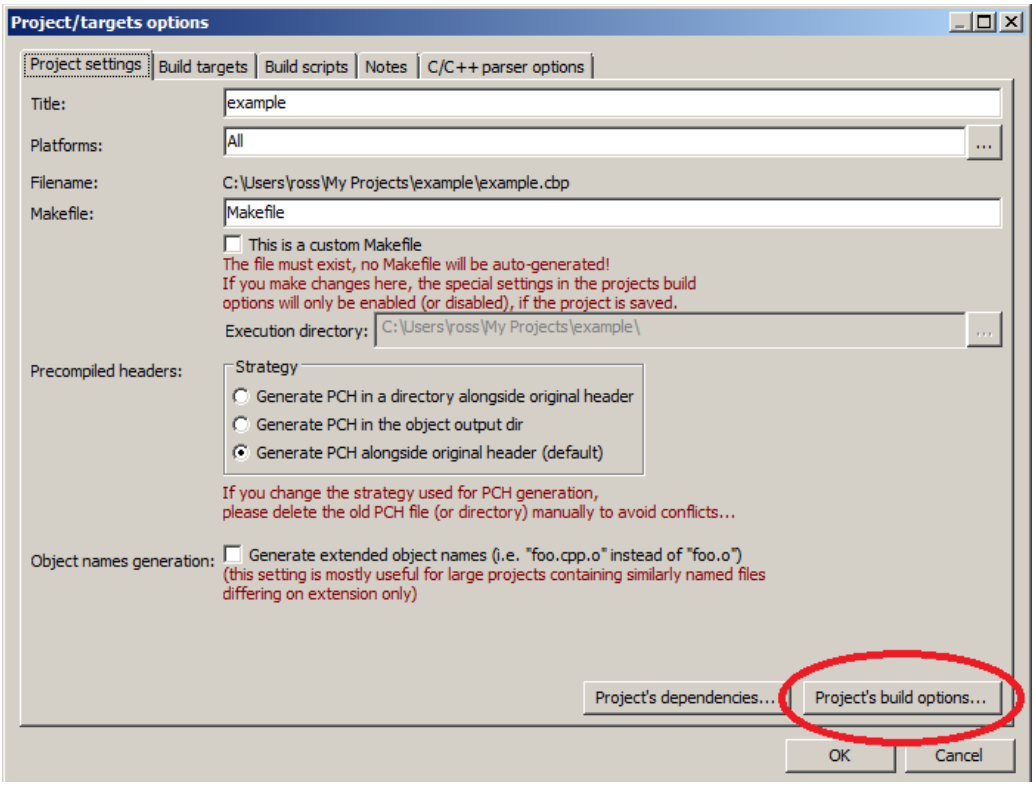


Press CTRL+D (or just close the terminal emulator window) to return to Code::Blocks. You now have C working on your Propeller!

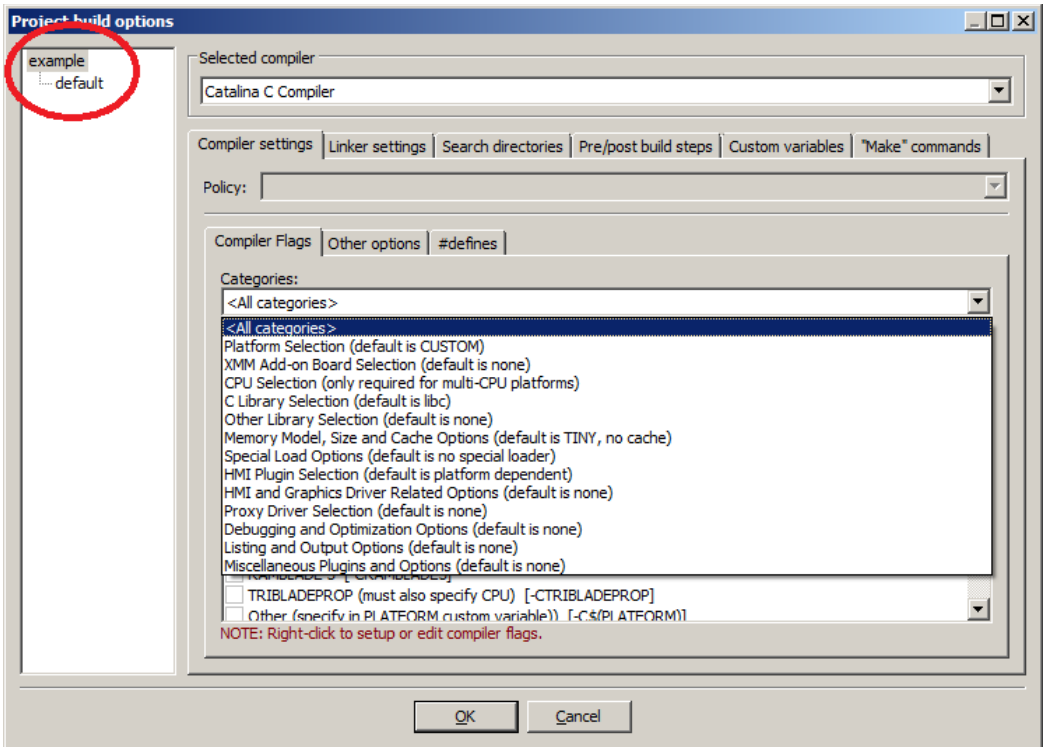
Changing your Catalina Project Options

The Catalina Project Wizard does not allow you to set all the available Catalina options, only the most commonly used ones.

If you want to change some options after your project has been created, or add options not supported by the Wizard, select **Project->Properties**, then select **Project's Build Options...** from the dialog box that appears:



Code::Blocks offers several other means of bringing up the project build options, but this one is *guaranteed* to result in the top-level Project node being selected in the resulting options dialog:



The reason this is important is that Code::Blocks actually has *two different* sets of options - one for the *project* (which applies to *all* build targets) and another for each *build target*.

If you make all your changes to the same set of options, Code::Blocks will check whether you have selected an internally consistent set of options (for example, it will not allow you to select two different propeller platforms at the same time, or two different HMI options, or the cache if you have selected the TINY or COMPACT memory models, or the maths library if you have specified NO_FLOAT).

This is a ***very powerful*** feature of Code::Blocks, and all you need to do to use it to the full is ensure you always select the build options via **Project->Preferences->Project's Build Options...**

Use the **Categories** dropdown to select the type of option you want to modify. All this does is filter the list of options down to a manageable size.

Just remember ... *always select the build options via*

Project->Preferences->Project's Build Options...

and Code::Blocks will check your option selections for you!

Manually Specifying Catalina Options

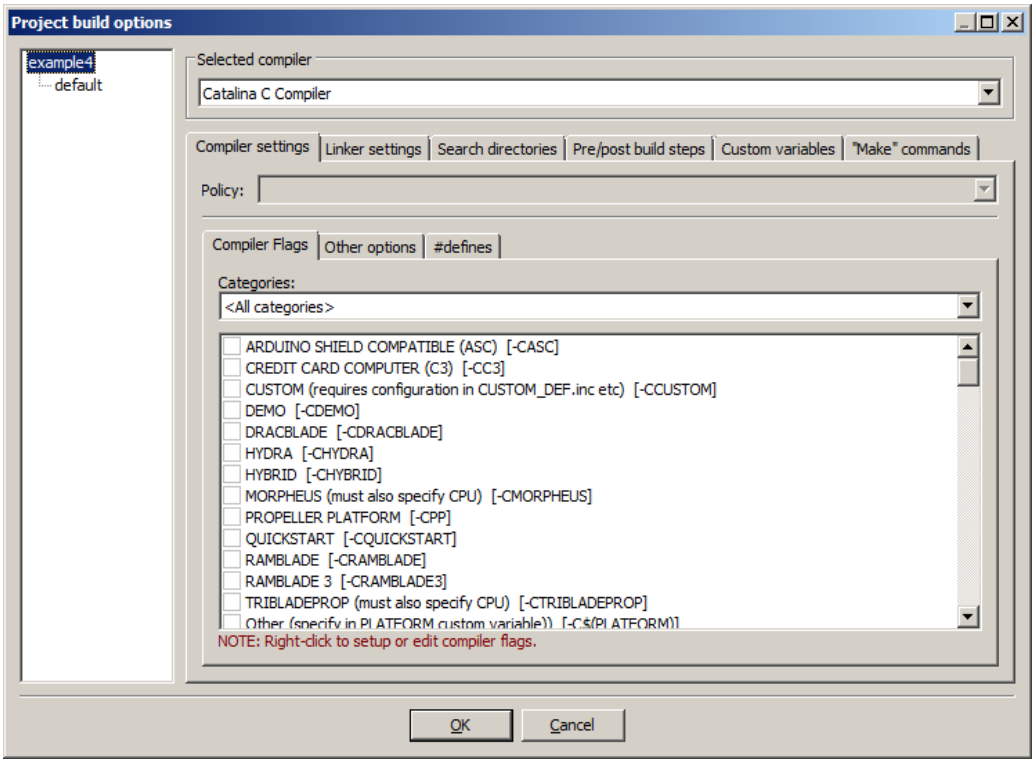
The **Catalina Project Wizard** sets up various basic compiler options for you, but there are many more options that you can set manually in your project. To see them all, select **Project->Preferences** and then **Project's Build Options ...**.

A dialog box similar to the one shown below will appear. The most important (and initially confusing) thing about setting options in Code::Blocks is that you can set them at the *Project* level, or at the level of each *Build Target* (e.g. *Release* or *Debug*).

It is generally a good idea to only set all options at the *Project* level unless there is a good reason to do otherwise - so whenever you open this dialog box, it is worthwhile getting into the habit of checking which node is selected, and selecting the *Project* node if it is not already selected:

The selected compiler for all projects created using the **Catalina Project Wizard** will be the **Catalina C Compiler**, and the **Compiler Flags** tab will show a complete list of Catalina compiler options. Some options will already have been selected by the Wizard.

The complete list of compiler options is far too cumbersome to be much use, so instead, choose a particular *category* of options from the **Categories** drop down list:

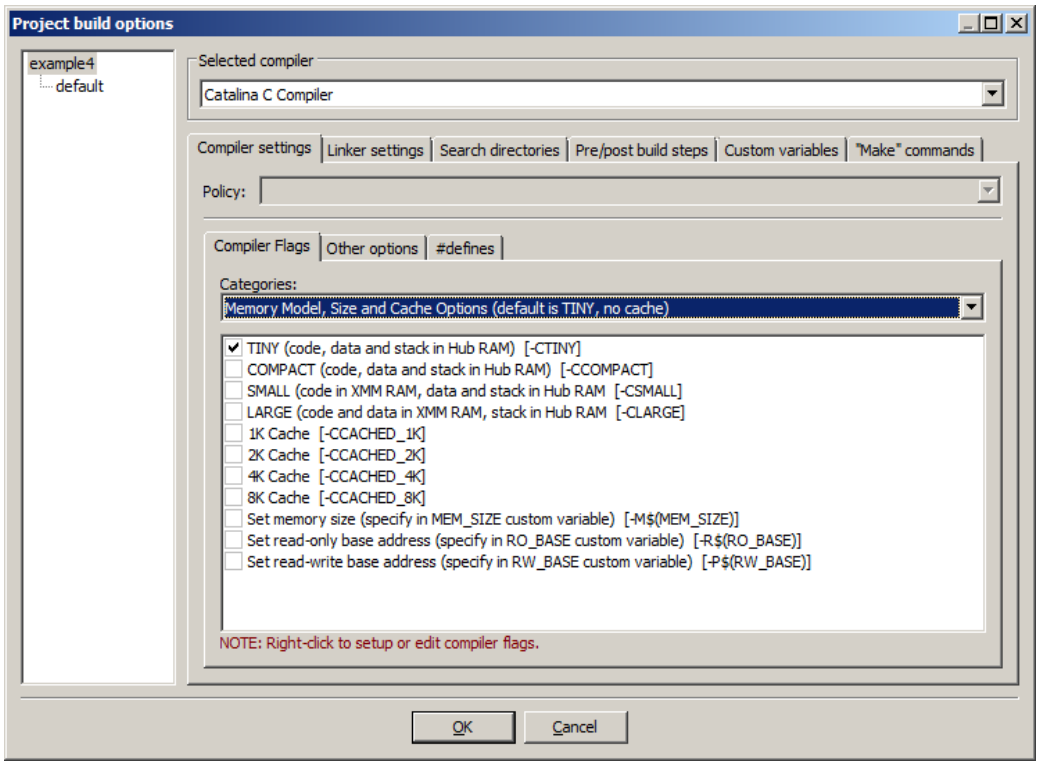


The categories are intended to group the available options by function. The categories are:

- Platform Selection
- XMM Add-on Board Selection
- CPU Selection
- C Library Selection

- Other Library Selection
- Memory Model, Size and Cache Options
- Special Load Options
- HMI Driver Selection
- HMI Related Options
- Proxy Driver Selection
- Kernel Options
- Debugging and Optimization Options
- Listing and Output Options
- Miscellaneous Options

Some of these options you will have already seen in the Wizard. Others are for advanced use only. The options are all the same as those used when invoking Catalina from a command line – but with Code::Blocks you don't have to remember them all, or what each one is for!.
For example, here are the options listed under the Memory Model, Size and Cache options:



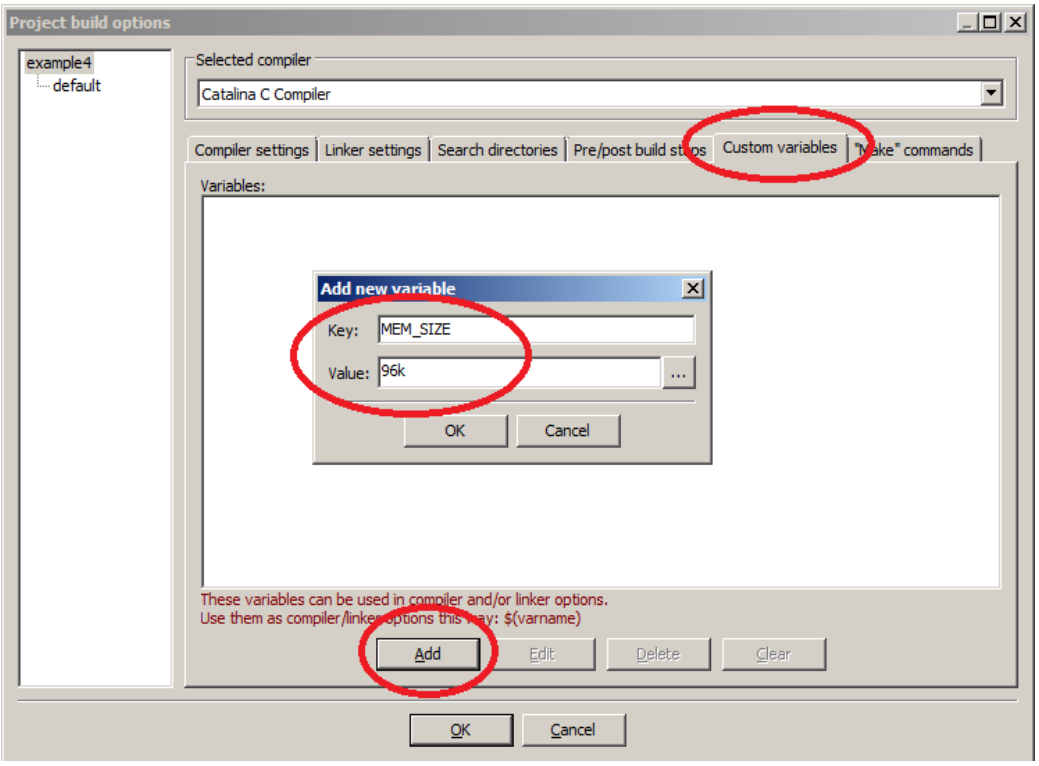
The **TINY** option is shown as selected because this is what we chose in the Wizard. If your Propeller platform has no XMM, this and **COMPACT** are the only options you will ever need to select from this category – the others are of interest for platforms with XMM RAM.

The Code::Blocks IDE tries to prevent you selecting multiple options if they are incompatible – e.g. here you cannot select both **TINY** and **LARGE** as your memory model. However, it is still possible to select incompatible options at the *Project* and *Build Target* levels, so if the build is not behaving as you expected, it is worth checking that options specified at different levels are not interfering with each other.

For more detail about each option, refer to the **Catalina Reference Manual** (the actual command-line option that corresponds to each selection is shown in square brackets).

In some cases, the option indicates that it requires a named variable which must be specified on the **Custom Variables** tab. An example in the category above would be the **Set Memory Size** option, which expects the actual memory size to be specified in a variable called **MEM_SIZE**. To use this option, you not only need to select it in this dialog box, you then need to go to the **Custom Variables** tab and enter both the appropriate variable and an appropriate value for it.

For example, to use the value **128k** as the **Memory Size**, you would define the **MEM_SIZE** variable on the **Custom Variable** tab as shown below:



Note that if there is no predefined option to do what you need, you can also specify compiler options by specifying them on the **Other Options** tab, or define arbitrary symbols by specifying them on the **#defines** tab (note that these will be normal C symbols, not Catalina symbols).

After changing any Compiler options, it is a good idea to rebuild the project and verify that the options are correctly specified in the command line shown in the build log. To do this, it may be useful to modify the default logging to show the actual command-line used. This is discussed in the next section.

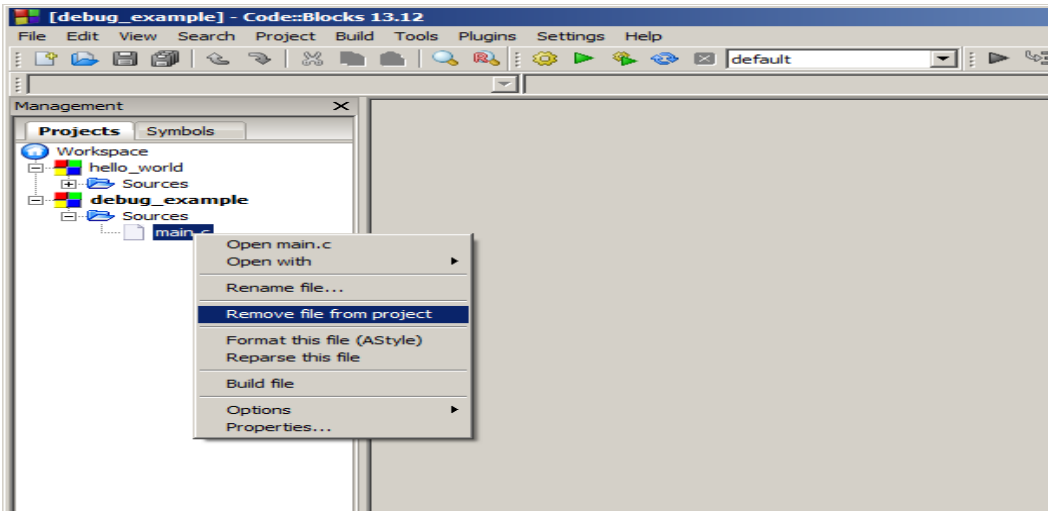
Building a more Complex Catalina Project

Now we are ready to build a more complex project. We will build a project using files that already exist in the Catalina **demos** directory, and also use this project to demonstrate the use of the **blackbox** debugger from within Code::Blocks.

You don't need to close the current project. Simply invoke the **Catalina Project Wizard** again (**File->New->Project...**) and specify **Debug_Example** as the name of the project (don't use spaces in the project name, as this name is also used as the name of the final executable).

Also in the wizard, select your Propeller platform (e.g. **C3**), but this time select the **Integer-Only C Library** (the **Standard C Library** will result in too large an executable on some platforms), the **TINY** memory model and one of the non-serial HMI options (such as **TV** or **VGA**).

When the new project has been built, it will again contain a default *main.c* source file, but we don't want that file – so right click on the file and select **Remove File From Project**:



Then, right click on the project node and select **Add Files …**

In the file file selection dialog box that appears, navigate to the Catalina debug example in the demo folder – e.g. (in Windows):

```
C:\Program Files\Catalina\demos\debug
```

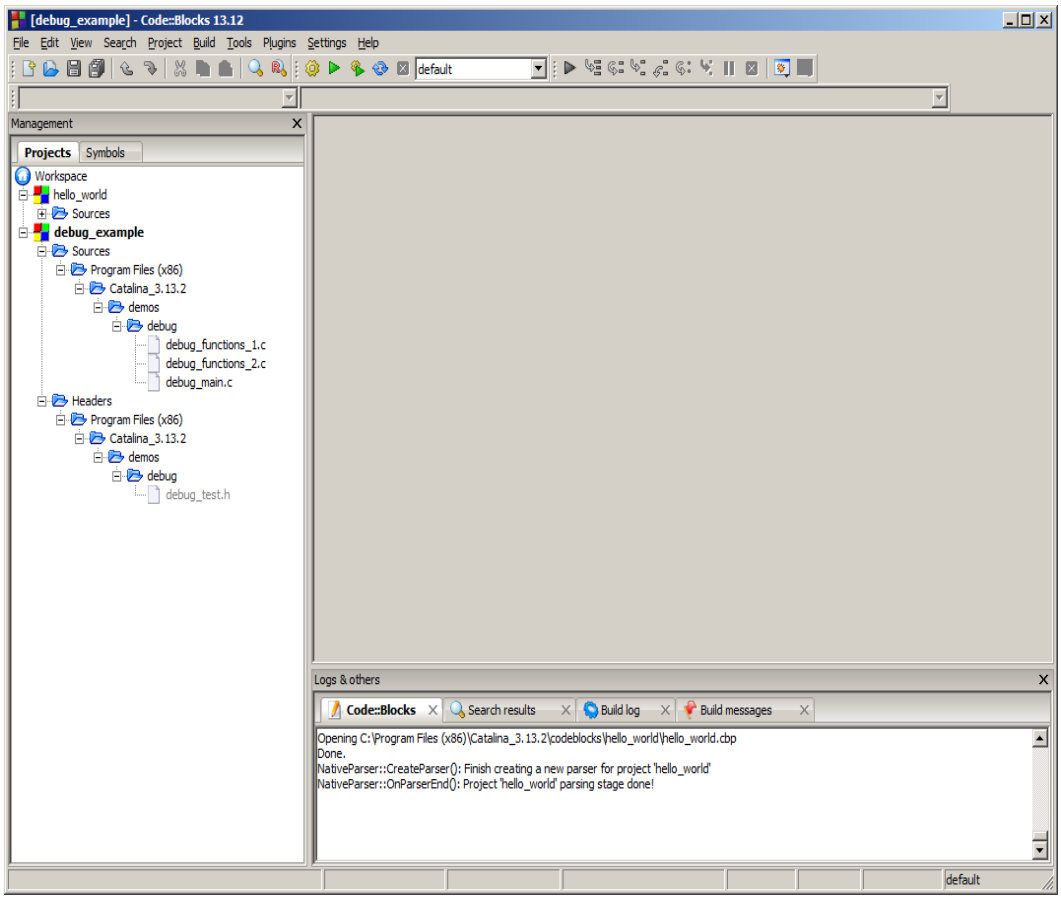
Or (in Linux):

```
/usr/local/lib/catalina/demos/debug
```

Then select the following four files (by holding down SHIFT while selecting each one):

```
debug_test.h
debug_functions_1.c
debug_functions_2.c
debug_main.c
```

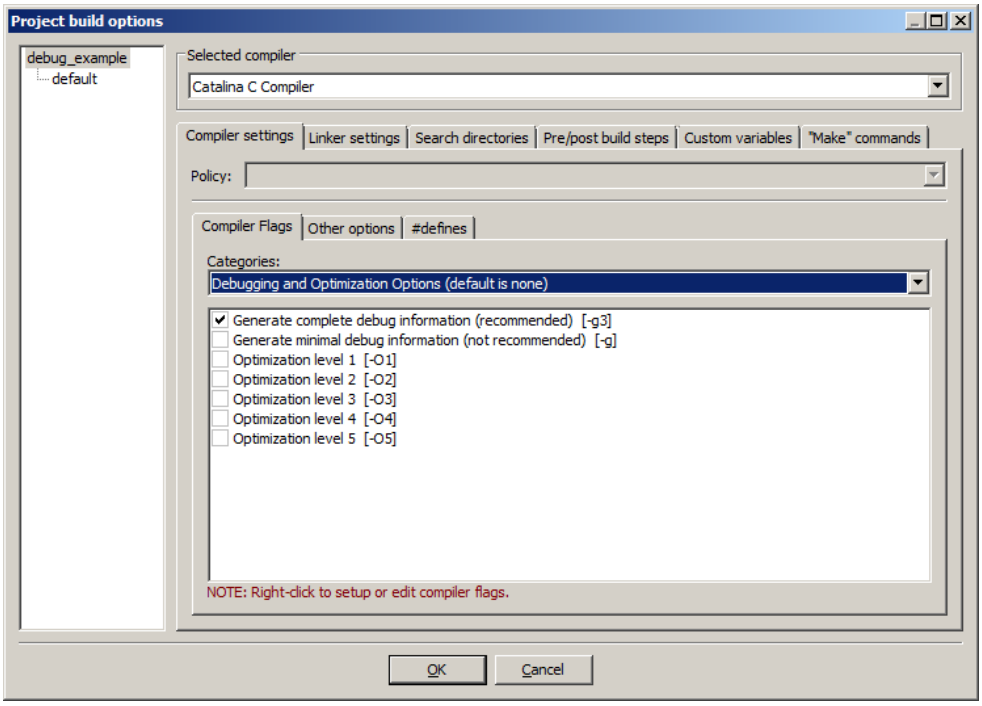
Now when you open the project node in the navigation pane, and keep opening each sub-node, you will end up with something like the following:



This is how Code::Blocks displays files that are included from a path outside the Project directory – these files have not been copied to the project directory – they still only exist in the demo folder – the project refers to them by their original path.

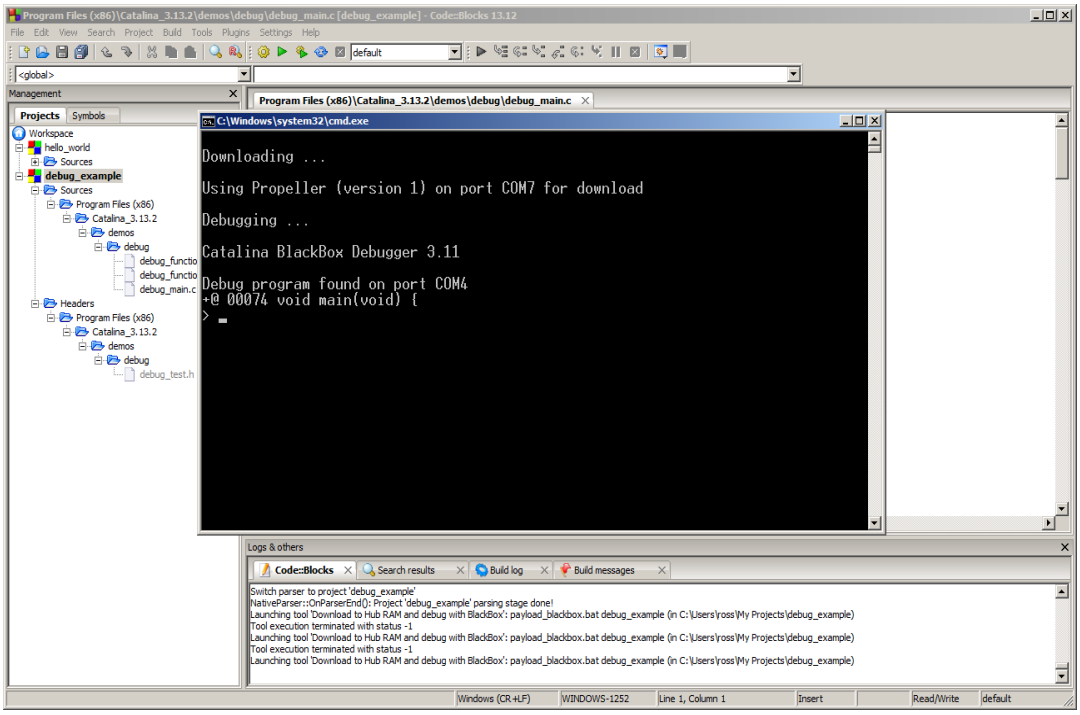
To test this project using the Catalina debugger, we must edit the compiler options to enable the debugger (I.e. add the **-g3** option to the project).

This can be selected by choosing **Project->Properties** from the main menu, then selecting **Project's Build Options …** then using the dialog box that appears to select the appropriate debugging option:



Now press **OK** and exit the dialog box. Then build the project (e.g. press **Ctrl-F9**).

Next, select **Tools->Download to Hub RAM and Debug with BlackBox** from the main menu[1]. A DOS command window (or a Linux command shell) will open, displayed in front of Code::Blocks, similar to the following:



[1] If the Tools menu is empty, see the section in this document entitled **If The Tools Menu is Empty**

This command box displays the progress of the **payload** loader, and is then used to run the **blackbox** debugger (initiated after the program has been downloaded). Once the debugger starts, press **n <ENTER>** a few times to step to the **next** line, and you should see the program output appear on whatever screen is connected to your Propeller. When you have finished debugging, press **e <ENTER>** to **exit** the debugger and return to Code::Blocks. Or at any time you can simply close the command window.

If you would prefer to use a graphical debugger instead, then (on Windows only) you could easily add another **Tool** to invoke the **BlackCat** debugger.

Select **Tools->Configure tools...** to see how to add another **Tool** to Code::Blocks.

Building The Catalina and Catalyst Demo Programs

All the Catalina and Catalyst demo programs can be built entirely within Code::Blocks. Example work spaces and projects for building many of them are provided.

If you installed Code::Blocks using the Windows Installer, menu entries for each workspace have been created in **Catalina -> Workspaces** group in the Windows Start Menu.

The following work spaces are provided for the Propeller 1:

demos	builds several of the simple programs from the demos folder.
smm_demos	demo programs built to use the SDCARD loader. These demos must be loaded from SD Card using Catalyst.
emm_demos	demo programs built to use the EEPROM loader. These demos must be programmed into an EEPROM, and note that some of these programs also use XMM.
xmm_demos	demo programs built to use XMM RAM.
proxy_demos	demo programs that use proxy drivers. These programs require a TriBladeProp or Morpheus.
debug_demos	programs from demos\debug
graphics_demos	programs from demos\graphics
multicog_demos	programs from demos\multicog
multithread_demos	programs from demos\multithread
spinc_demos	programs from demos\spinc
serial4_demos	programs from demos\serial4
catalyst_demos	programs from the catalyst folder
vgraphics_demos	programs from demos\vgraphics

The following work spaces are provided for the Propeller 2:

P2_demos	builds several of the simple programs from the demos folder for the Propeller 2.
P2_serial2_demos	programs from demos\serial2 for the Propeller 2.

To use each workspace, simply select on the file in the Windows Start menu, or open the workspace file in the Catalina *codeblocks* sub-directoty (e.g. **demos.workspace**) from within Code::Blocks.

Note that unless otherwise specified, none of the demo projects or work spaces have the **CUSTOM** Propeller platform specified. This means that they will compile using the **CUSTOM** platform setting unless you specify another platform to use in the global **Compiler Settings**, or in the individual project **Build Options**.

Notes about the Catalina Tools

General Note

The Catalina items in the **Tools** menu will not work until you have a Catalina C Project open in Code::Blocks. This is because the path to the executable tools is not specified in the tools themselves – it is only set once you have specified the Compiler to use.

If the Tools Menu is empty

The Code::Blocks **Tools** menu may be empty (apart from a single **Configure Tools...** entry) after installing Catalina and Code::Blocks via the Windows installer. This situation can occur when installing Catalina as a user that does *not* have administrator rights. In this case, you will have to manually set up the Code::Blocks *default.conf* file for that user (which is where the menu entries are stored). You will also have to set this up for *each additional user* on the same PC that wants to use Code::Blocks.

You can do this by opening up a **Catalina Command Line** window and executing the batch file **Reset_Codeblocks**. You may need to do this as each user, and you may run the command-line window as Administrator (right click on the Start Menu entry for the **Catalina Command Line** and select “Run as Administrator”).

If a program compiles and loads but won't run on the Propeller 1

There is a potential problem if you build *the same project* for both the Propeller 1 and the Propeller 2. This can occur because you may end up with both a **.bin** and a **.binary** file in the output directory, which may not be deleted even if you “Clean” the project.

If this occurs, the payload loader will always first try to load the **.bin** file (if it exists), not the **.binary** – which will be the wrong file for a Propeller 1 (**.bin** files are only intended for the Propeller 2).

If you suspect this is the case you can check the output of the payload command in the Code::Blocks status/output window to see the directory where the program is being loaded *from*. Then manually go to that location, check if there is a **.bin** version of the file there, and delete it.

The Build XMM Utilities Tool

One of the items in the **Tools** menu represents a utility developed specifically to simplify the use of Code::Blocks in conjunction with Propeller platforms with XMM RAM. This is the **Build XMM Utilities** menu item.

When you invoke this tool (make sure you have your Catalina XMM project open first!) you will be guided through a set of questions much like the **Catalina Project Wizard**. The purpose of these questions is to build the appropriate XMM utilities necessary to download and debug programs on the XMM platform you are using. Before you use any of the other XMM tools, you will need to invoke this tool (*once)* when you build projects for a Propeller platform with XMM RAM. If you subsequently need to download or debug projects on a *different* XMM platform, you should re-

invoke the tool. Also, note that your Catalina Project must use the *same options* as you specify in this tool, or the other XMM RAM tools will not work correctly.

Environment Variables and Code::Blocks

The environment variables used by the Catalina command-line compiler also apply when Catalina is invoked by Code::Blocks. If you set these environment variables at the system or user level[2] then this this may cause unwanted conflicts with the Catalina options set in your Code::Blocks project.

If in doubt, enable logging of the full command line in Code::Blocks, and also add the Verbose (**-v)** option to Catalina to list what options are actually in effect when building your project.

However, if you installed Catalina to a non-standard location, setting at least the **LCCDIR** environment variable at the system level is very useful. It not only simplifies usage of Catalina via the command-line, it also allows Code::Blocks to auto-detect where Catalina is installed.

Also, when Code::Blocks successfully auto-detects Catalina, it will import the current settings of the following two environment variables. They will be configured as the default search directories in the Code::Blocks Compiler configuration. The purpose of this is to try and make projects built in Code::Blocks continue to build correctly even if you subsequently change these variables for using Catalina in command-line mode:

CATALINA_INCLUDE

CATALINA_LIBRARY

These settings can be manually modified later in the **Search Directories** tab of the **Settings->Compiler and Debugger...** dialog box if required.

Code::Blocks Limitations

Code::Blocks was initially developed for self-hosted development. While it is increasingly being used for cross compilation and embedded development its roots are sometimes still evident.

The Code::Blocks Debugger plugin does not currently support the Catalina debuggers - the **blackbox** and **blackcat** debuggers can be *launched* from Code::Blocks, but will run in separate windows. This may be added in a future release of Catalina, but at present it is probably best to disable or remove the Code::Blocks Debugger plugin altogether (unless you also use it in conjunction with other compilers). The version of Code::Blocks installed by the Windows installer does not include the debugger plugin, although it can be installed manually if required.

What's Next with Code::Blocks?

Code::Blocks is a very sophisticated IDE, and has many more features than have been discussed here. For more information, consult the Code::Blocks documentation.

---

[2] For example, in Windows you do this from **System Properties** - e.g. in Windows 7, select **Start->Control Panel->System and Security->Advanced System Settings** (the precise location differs in other versions of Windows). In the resulting dialog box, select the **Advanced** Tab and press **Environment Variables**.

Also, there are many additional plugins available for Code::Blocks that are designed to assist in the development of C programs. These plugins will generally work perfectly well with Catalina.